Pitch: Triagem Seletiva para Escalar a Curadoria de Benchmarks de LLMs

A ideia em uma frase

Queremos reduzir o número de perguntas que precisam de inspeção humana durante a limpeza de benchmarks de LLMs, usando as respostas já existentes na cache para calcular um score de confiança e encaminhar apenas os casos incertos para revisão.

O problema

O artigo *Do Large Language Model Benchmarks Test Reliability?* mostra que benchmarks populares contêm perguntas mal formuladas, ambíguas ou com respostas erradas. Para criar os *Platinum Benchmarks*, os autores deram as perguntas a vários modelos e inspecionaram manualmente os casos em que algum modelo falhou ou discordou.

Este processo funciona, mas não escala bem. Se tivermos milhares ou milhões de perguntas, não é realista pedir a especialistas para verificarem manualmente todas as instâncias sinalizadas.

Existe ainda um ponto cego: se todos os modelos derem a mesma resposta, uma pergunta problemática pode não ser sinalizada. O nosso projeto não promete resolver completamente esse ponto cego; vai medir e reduzir de forma controlada o problema principal da escalabilidade.

A proposta

Vamos construir um **auditor seletivo offline** sobre o repositório oficial `MadryLab/platinum-benchmarks`.

Para cada pergunta, o auditor:

1. lê as respostas dos modelos nos ficheiros `.pkl` da cache;
2. extrai a resposta final de cada texto;
3. mede a força da maioria e a entropia dos votos;
4. mede a estabilidade das justificações;
5. calcula um score entre 0 e 1;
6. aceita automaticamente os casos suficientemente seguros;
7. envia os casos incertos para inspeção humana.

O objetivo não é substituir os revisores. É transformar revisão total numa revisão seletiva e mensuravelmente mais barata.

```
Perguntas + respostas na cache
              |
              v
      Auditor seletivo
              |
      +-------+-------+
```

```
      |              |
      v              v
 Aceitar          Deferir
 automaticamente  para revisão humana
```

Exemplo prático

Pergunta de matemática:

```
Uma quinta tem 3 caixas com 12 maçãs cada.
Oferece 7 maçãs. Quantas maçãs restam?
```

A resposta correta é 29, mas o auditor não usa essa resposta para calcular o score. Observa apenas as respostas existentes:

```
Modelo A -> "3 x 12 = 36; 36 - 7 = 29. Answer: 29"
Modelo B -> "Initially there are 36. After giving away 7: 29."
Modelo C -> "There are 29 apples left. Answer: 29"
Modelo D -> "3 x 12 = 36. Answer: 36"
Modelo E -> "The answer is 29."
```

Depois do parser do repositório:

```
29, 29, 29, 36, 29
```

O auditor vê uma maioria de 4 em 5, duas respostas distintas e quatro justificações compatíveis. O score será intermédio ou alto, dependendo da estabilidade textual. Se o limiar for alto, o caso é deferido; se for mais permissivo, pode ser aceite.

Outro exemplo:

```
Modelo A -> 29
Modelo B -> 29
Modelo C -> 29
Modelo D -> 36
Modelo E -> 22
```

Aqui a maioria é fraca e há três respostas diferentes. A entropia é alta, pelo que o caso deve ser enviado para revisão.

O que entra e não entra no score

Entram no score a resposta final extraída, a distribuição de votos, a força da maioria, a entropia, o número de respostas distintas, a similaridade textual entre justificações, a estabilidade do comprimento, indicadores exploratórios de hesitação e a completude da cache.

O `platinum_target`, a resposta de referência do benchmark, **não entra no score de decisão**. Usá-lo diretamente seria *label leakage*. É usado apenas depois para medir se a aceitação estava correta, calibrar o limiar no desenvolvimento e calcular risco, cobertura e AURC.

Como o score é calculado

Se quatro dos cinco modelos respondem 29:

```
majority_ratio = 4 / 5 = 0,80
```

Para os votos $v_1, \ldots, v_k$, calculamos:

> **$H(x) = -\Sigma\, p(c \mid x) \times \log p(c \mid x)$**

> **$\text{VoteConfidence}(x) = 1 - H(x) / \log(K)$**

Unanimidade dá confiança próxima de 1; votos equilibrados dão confiança baixa.

Quando possível, separamos `<cot>...</cot>` ou `<thinking>...</thinking>`. Caso não existam, removemos a secção a partir de `Answer:`. Se isso falhar, usamos o texto completo e registamos o fallback.

Cada justificação é convertida para TF-IDF. Calculamos a similaridade coseno entre todos os pares e fazemos a média:

> **$\text{TextSimilarity}(x) = \text{média}\ [\cos(\text{TFIDF}(R_i), \text{TFIDF}(R_j))]\ \text{para todos os pares}\ i < j$**

Isto mede estabilidade textual, não prova correção lógica. Explicações semelhantes podem repetir o mesmo erro.

O score combinado baseline será:

> **$S(x) = 0{,}40 \times \text{MajorityRatio}(x) + 0{,}35 \times \text{VoteConfidence}(x) + 0{,}15 \times \text{TextSimilarity}(x) + 0{,}10 \times \text{LengthStability}(x)$**

Com um limiar inicial ilustrativo $\tau = 0{,}80$:

Situação	Score típico	Decisão
5/5 com justificações estáveis	0,90–1,00	Aceitar automaticamente
4/5, textos coerentes	0,75–0,90	Caso fronteira
3/5, respostas divergentes	0,50–0,70	Deferir
2/5 ou várias respostas distintas	0,00–0,50	Deferir
Cache incompleta	não aplicável	Deferir obrigatoriamente

O valor final de τ será escolhido no desenvolvimento para uma meta como “risco máximo de 5% nos casos aceites” e ficará congelado na avaliação final.

O que vamos comparar

1. **Revisão total:** todas as perguntas são inspeccionadas por humanos.
2. **Maioria simples:** aceita-se sempre a resposta mais votada.
3. **Auditor seletivo:** só os casos abaixo do limiar são enviados para revisão.

Métricas: cobertura, redução de revisão, risco seletivo, curva risco-cobertura, AURC e precisão/recall dos casos deferidos.

Estratégia	Inspeções humanas	Aceites automáticos	Erros entre aceites
Revisão total	1.000	0	—
Maioria simples	0	1.000	80
Auditor seletivo	250	750	20

Os números são ilustrativos; os resultados reais virão da cache.

O limite dos consensos errados

Se cinco modelos responderem `36` quando o alvo é `29`, teremos:

```
36, 36, 36, 36, 36
```

O score será alto porque existe unanimidade. Isto é um limite fundamental de observar apenas as respostas originais: não conseguimos distinguir consenso correto de erro correlacionado sem evidência adicional.

O projeto não promete detetar todas as perguntas erradas. Mede quanto trabalho humano pode ser poupado mantendo um risco conhecido e controlado.

Experimento secundário

Depois do Arm 1, selecionamos no máximo 20 perguntas aceites automaticamente, sobretudo em `DROP`, onde o artigo descreve o *first-event bias*.

Aplicamos uma transformação temporal controlada, como trocar “o que aconteceu primeiro?” por “o que aconteceu segundo?”, quando a resposta for verificável, e uma transformação placebo, como acrescentar uma frase neutra.

As novas perguntas não estão na cache original. Este pequeno stress test requer um modelo local leve ou cache adicional. É uma validação de robustez, não a contribuição principal, e não bloqueia a conclusão do Arm 1.

Plano de trabalho

Estudante A — dados: inventário dos Pickle, loader offline, integração com o dataset, parsers e matriz de respostas.

Estudante B — auditor: entropia, score, calibração, risco-cobertura, AURC, baselines, gráficos e tabelas.

Estudante C — análise: consenso operacional, stress test temporal/placebo, falsos negativos e limitações.

Todos participam na revisão de literatura, interpretação e apresentação.

Cronograma

- **Semana 1 / Checkpoint 1:** artigo, limitação, literatura curta, cache e primeira tabela; pitch de 3 minutos.
- **Semana 2 / Checkpoint 2:** score, baselines, resultados preliminares em `gsm8k`, `svamp` e `drop`; apresentação de 5 minutos.
- **Semana 3:** congelar protocolo, resultados finais do Arm 1 e stress test secundário se o ambiente local estiver preparado.
- **Semana 4:** execução limpa, notebook, README, slides e submissão.

Mensagem final

A contribuição principal é medir se um auditor seletivo consegue reduzir a inspeção humana sem aumentar demasiado o risco. Mesmo que o score não supere a maioria simples, o projeto continua válido: terá medido uma limitação real, identificado sinais insuficientes e explicado por que consenso e estabilidade textual não garantem correção.